Extended Transition Function

Transition Function, $\delta : Q \times \Sigma \rightarrow Q$

It is a mapping that determines the next state of a finite automaton (FA) based on the current state and the current input symbol.

For example, $\delta(q_2, a) = q_3$

Here $Q = \{q_1, q_2, \ldots\}$ and $\Sigma = \{a, b, \ldots\}$

Extended Transition Function, $\delta^\wedge$ or $\delta^* : Q \times \Sigma^* \rightarrow Q$

It is a mapping that determines the next state of a finite automaton (FA) based on the current state and the current input sequence.

Basis: $\delta^*(q, \varepsilon) = q$

Induction: $\delta^*(q, w) = \delta(\delta^*(q, x), a)$ where $w = xa$

Extended Transition Function

Extended Transition Function Example:
Let for the string: 1010

$\delta^*(q_0, \varepsilon) = q_0$ // q_0 is the start state

$\delta^*(q_0, 1) = \delta(\delta^*(q_0, \varepsilon), 1) = \delta(q_0, 1) = q_1$

$\delta^*(q_0, 10) = \delta(\delta^*(q_0, 1), 0) = \delta(q_1, 0) = q_2$

$\delta^*(q_0, 101) = \delta(\delta^*(q_0, 10), 1) = \delta(q_2, 1) = q_3$

$\delta^*(q_0, 1010) = \delta(\delta^*(q_0, 101), 0) = \delta(q_3, 0) = q_4$

$\delta^*(q_0, 1010) = q_4$ // If q_4 is an accepting state then the string will be accepted otherwise not.

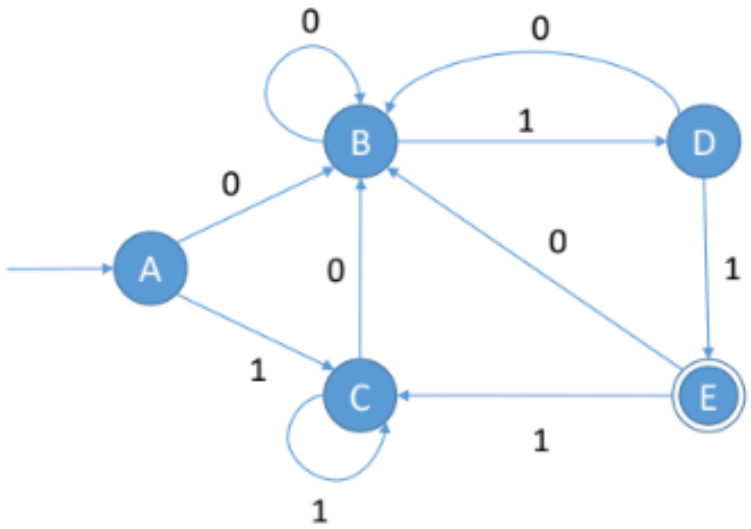
Table Filling Algorithm

Equivalence: p and q said to be equivalence if for all string w, $\delta^*(p, w)$ and $\delta^*(q, w)$ gives the accepting state or non-accepting state.

Distinguishable: p and q said to be distinguishable if for at least one string w, One of $\delta^*(p, w)$ and $\delta^*(q, w)$ gives the accepting state and another gives non-accepting state.

Table Filling Algorithm

Partitioning Method



0 Equivalence: {A, B, C, D} {E}
1 Equivalence: {A, B, C} {D} {E}
2 Equivalence: {A, C} {B} {D} {E}
3 Equivalence: {A, C} {B} {D} {E}

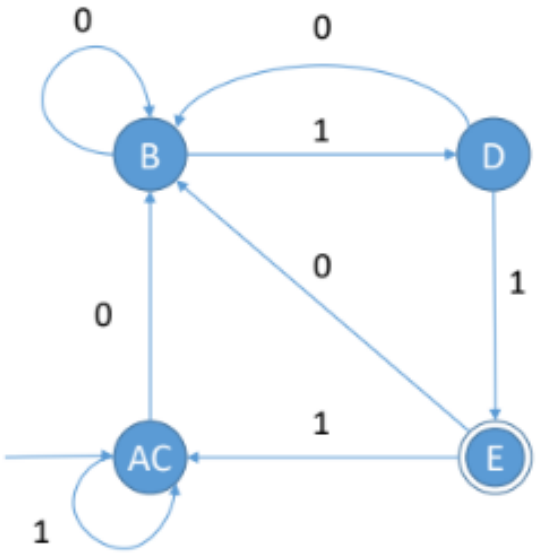
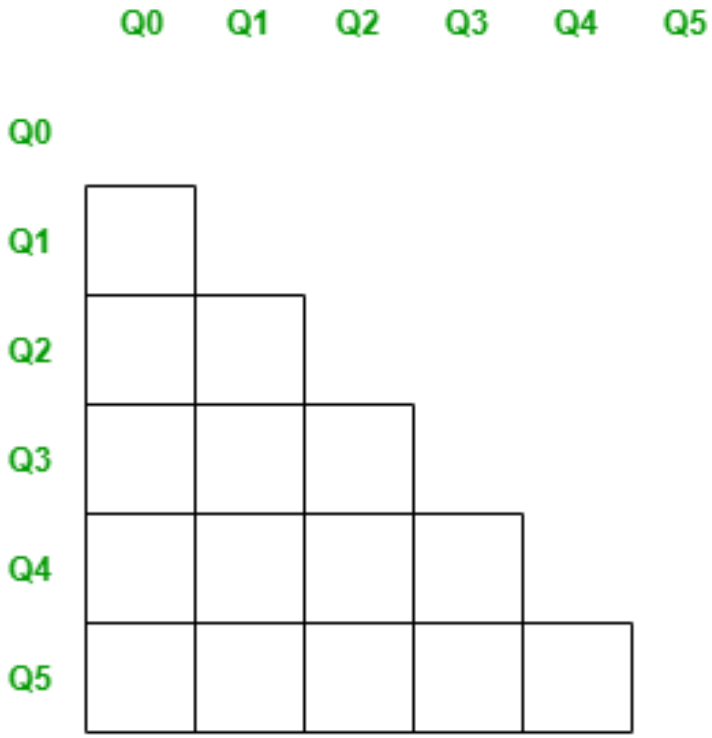


Table Filling Algorithm

Myhill-Nerode Theorem or Table Filling Method

Basis: If Q0 is an accepting state and Q1 is non-accepting state then the pain {Q0, Q1} is distinguishable.

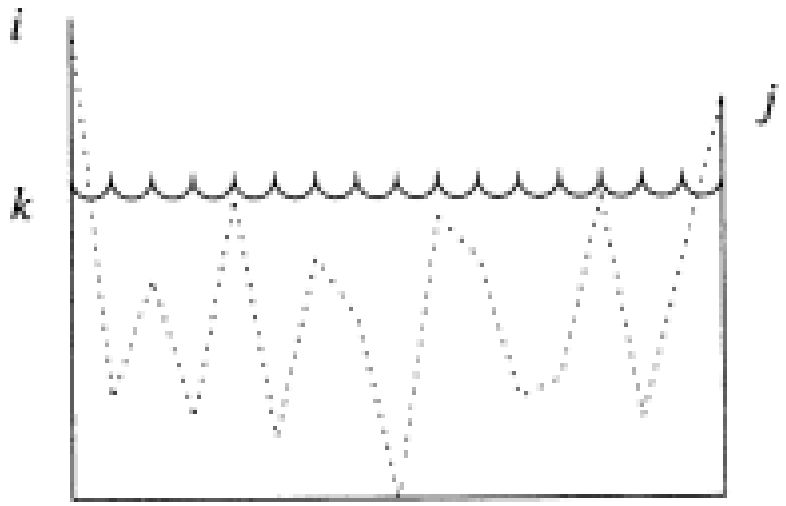
Induction: Let for some input symbol a, Q0 and Q1 gives output accordingly $\delta(Q0, a) = Q2$ and $\delta(Q1, a) = Q3$.
If Q2 and Q3 are distinguishable then Q0 and Q1 also be distinguishable.



$R_{ij}^{(k)}$ Method

Let us suppose that A's states are {1, 2,…,n} for some integer n.

$R_{ij}^{(k)}$ as the name of a regular expression whose language is the set of strings w such that w is the label of a path from state i to state j in A, and that path has no intermediate node whose number is greater than k.



$R_{ij}^{(k)}$ Method

BASIS: The basis is $k = 0$. Since all states are numbered 1 or above, the restriction on paths is that the path must have no intermediate states at all. There are only two kinds of paths that meet such a condition:

1. An arc from node (state) i to node j.
2. A path of length 0 that consists of only some node i.

If $i \neq j$, then only case (1) is possible. We must examine the DFA A and find those input symbols a such that there is a transition from state i to state j on symbol a.

a) If there is no such symbol a, then $R_{ij}^{(0)} = \emptyset$.

b) If there is exactly one such symbol a, then $R_{ij}^{(0)} = \mathbf{a}$.

c) If there are symbols $a_1, a_2, \ldots, a_k$ that label arcs from state i to state j, then $R_{ij}^{(0)} = \mathbf{a}_1 + \mathbf{a}_2 + \cdots + \mathbf{a}_k$.

However, if $i = j$, then the legal paths are the path of length 0 and all loops from i to itself. The path of length 0 is represented by the regular expression ϵ, since that path has no symbols along it. Thus, we add ϵ to the various expressions devised in (a) through (c) above. That is, in case (a) [no symbol a] the expression becomes ϵ, in case (b) [one symbol a] the expression becomes $\epsilon + \mathbf{a}$, and in case (c) [multiple symbols] the expression becomes $\epsilon + \mathbf{a}_1 + \mathbf{a}_2 + \cdots + \mathbf{a}_k$.

$R_{ij}^{(k)}$ Method

INDUCTION: Suppose there is a path from state i to state j that goes through no state higher than k. There are two possible cases to consider:

1. The path does not go through state k at all. In this case, the label of the path is in the language of $R_{ij}^{(k-1)}$.

2. The path goes through state k at least once. Then we can break the path into several pieces, as suggested by Fig. 3.3. The first goes from state i to state k without passing through k, the last piece goes from k to j without passing through k, and all the pieces in the middle go from k to itself, without passing through k. Note that if the path goes through state k only once, then there are no "middle" pieces, just a path from i to k and a path from k to j. The set of labels for all paths of this type is represented by the regular expression $R_{ik}^{(k-1)}(R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}$. That is, the first expression represents the part of the path that gets to state k the first time, the second represents the portion that goes from k to itself, zero times, once, or more than once, and the third expression represents the part of the path that leaves k for the last time and goes to state j.

$R_{ij}^{(k)}$ Method

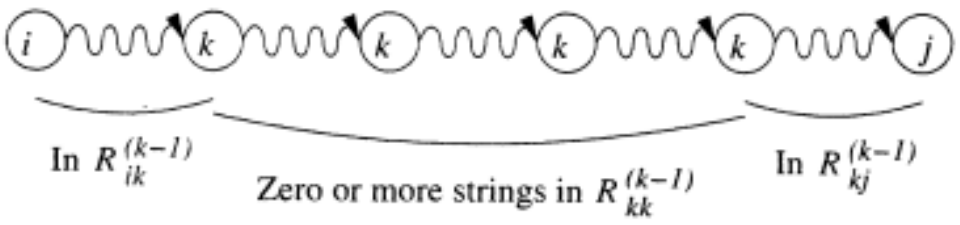


Figure 3.3: A path from i to j can be broken into segments at each point where it goes through state k

When we combine the expressions for the paths of the two types above, we have the expression

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)}(R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}$$

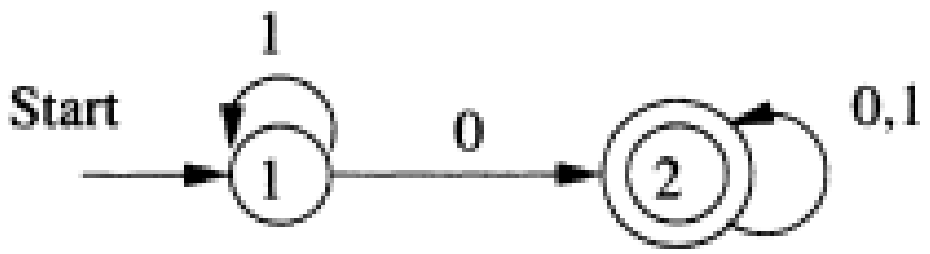
for the labels of all paths from state i to state j that go through no state higher than k. If we construct these expressions in order of increasing superscript, then since each $R_{ij}^{(k)}$ depends only on expressions with a smaller superscript, then all expressions are available when we need them.

Eventually, we have $R_{ij}^{(n)}$ for all i and j. We may assume that state 1 is the start state, although the accepting states could be any set of the states. The regular expression for the language of the automaton is then the sum (union) of all expressions $R_{1j}^{(n)}$ such that state j is an accepting state. □

$R_{ij}^{(k)}$ Method

Basis

$R_{11}^{(0)}$	$\epsilon + \mathbf{1}$
$R_{12}^{(0)}$	$\mathbf{0}$
$R_{21}^{(0)}$	$\emptyset$
$R_{22}^{(0)}$	$(\epsilon + \mathbf{0} + \mathbf{1})$

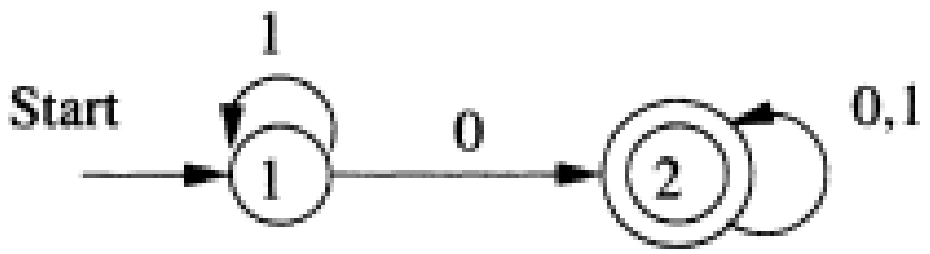


$R_{ij}^{(k)}$ Method

Induction

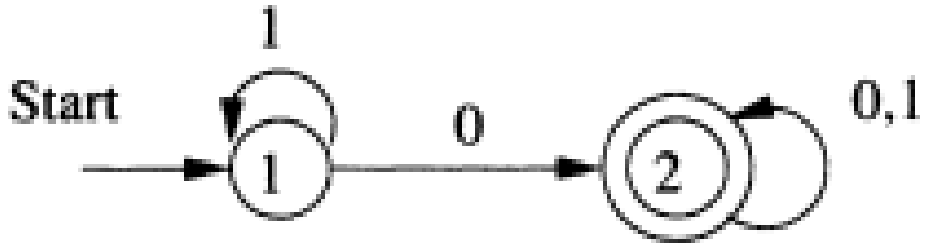
$$R_{ij}^{(1)} = R_{ij}^{(0)} + R_{i1}^{(0)}(R_{11}^{(0)})^* R_{1j}^{(0)}$$

	By direct substitution	Simplified
$R_{11}^{(1)}$	$\epsilon + 1 + (\epsilon + 1)(\epsilon + 1)^*(\epsilon + 1)$	1^*
$R_{12}^{(1)}$	$0 + (\epsilon + 1)(\epsilon + 1)^* 0$	$1^* 0$
$R_{21}^{(1)}$	$\emptyset + \emptyset(\epsilon + 1)^*(\epsilon + 1)$	$\emptyset$
$R_{22}^{(1)}$	$\epsilon + 0 + 1 + \emptyset(\epsilon + 1)^* 0$	$\epsilon + 0 + 1$



$R_{ij}^{(k)}$ Method

Induction



$$R_{ij}^{(2)} = R_{ij}^{(1)} + R_{i2}^{(1)}(R_{22}^{(1)})^* R_{2j}^{(1)}$$

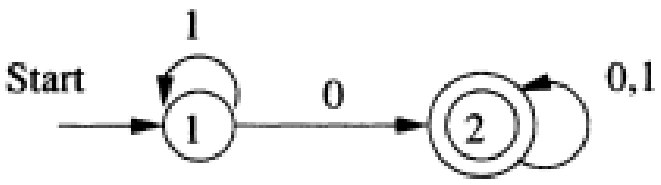
	By direct substitution	Simplified
$R_{11}^{(2)}$	$1^* + 1^*0(\epsilon + 0 + 1)^*\emptyset$	1^*
$R_{12}^{(2)}$	$1^*0 + 1^*0(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$1^*0(0 + 1)^*$
$R_{21}^{(2)}$	$\emptyset + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*\emptyset$	$\emptyset$
$R_{22}^{(2)}$	$\epsilon + 0 + 1 + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$(0 + 1)^*$

The final regular expression equivalent to the automaton of Fig. 3.4 is constructed by taking the union of all the expressions where the first state is the start state and the second state is accepting. In this example, with 1 as the start state and 2 as the only accepting state, we need only the expression $R_{12}^{(2)}$. This expression is $1^*0(0 + 1)^*$. It is simple to interpret this expression. Its language consists of all strings that begin with zero or more 1's, then have a 0,

$R_{ij}^{(k)}$ Method

Final Result

	By direct substitution	Simplified
$R_{11}^{(2)}$	$1^* + 1^*0(\epsilon + 0 + 1)^*\emptyset$	1^*
$R_{12}^{(2)}$	$1^*0 + 1^*0(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$1^*0(0 + 1)^*$
$R_{21}^{(2)}$	$\emptyset + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*\emptyset$	$\emptyset$
$R_{22}^{(2)}$	$\epsilon + 0 + 1 + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$(0 + 1)^*$



The final regular expression equivalent to the automaton of Fig. 3.4 is constructed by taking the union of all the expressions where the first state is the start state and the second state is accepting. In this example, with 1 as the start state and 2 as the only accepting state, we need only the expression $R_{12}^{(2)}$. This expression is $1^*0(0 + 1)^*$. It is simple to interpret this expression. Its language consists of all strings that begin with zero or more 1's, then have a 0, and then any string of 0's and 1's. Put another way, the language is all strings of 0's and 1's with at least one 0. □

NFA to DFA

Same as ε –NFA to DFA.
The Subset construction method

Reverse of DFA/NFA

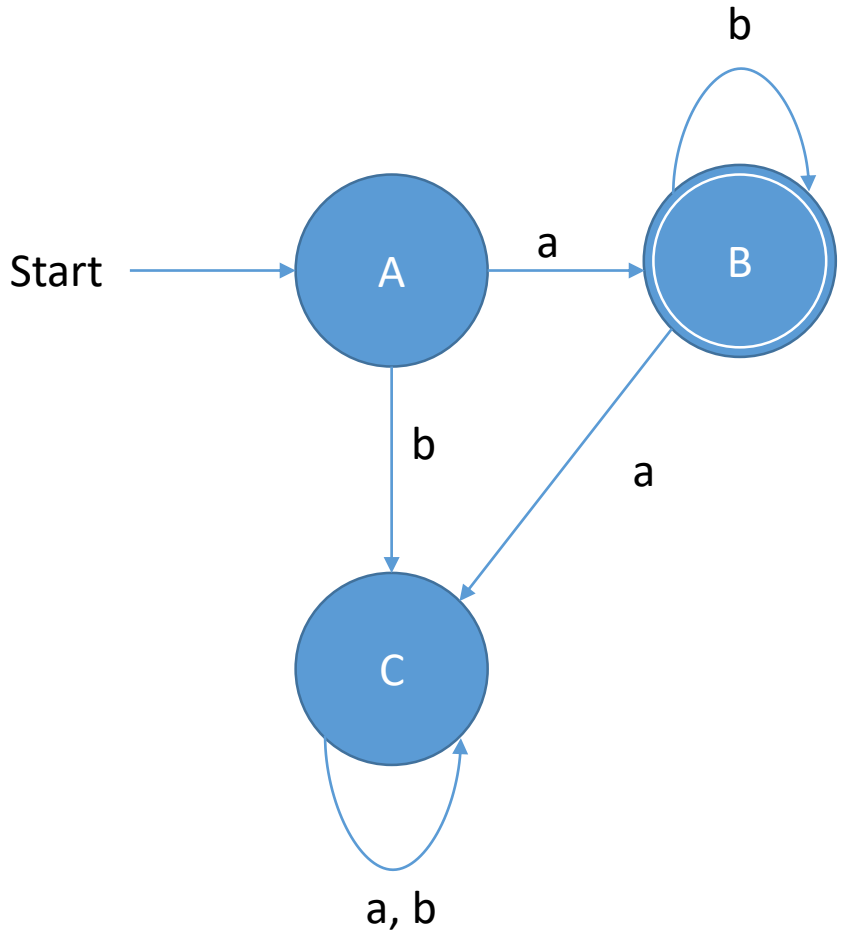
Reversal: We define the reversed language L^R of L to be the language $L^R = \{w^R \mid w \in L\}$, where $w^R := a_n a_{n-1} \ldots a_1 a_0$ for $w = a_0 a_1 \ldots a_{n-1} a_n$

Steps to Reversal:

1. Draw the states as it is.
2. Add a new single accepting state
3. Make the accepting state the initial state and initial state the accepting state
4. Invert the transition function into a transition relation
5. Remove any states that are no longer necessary

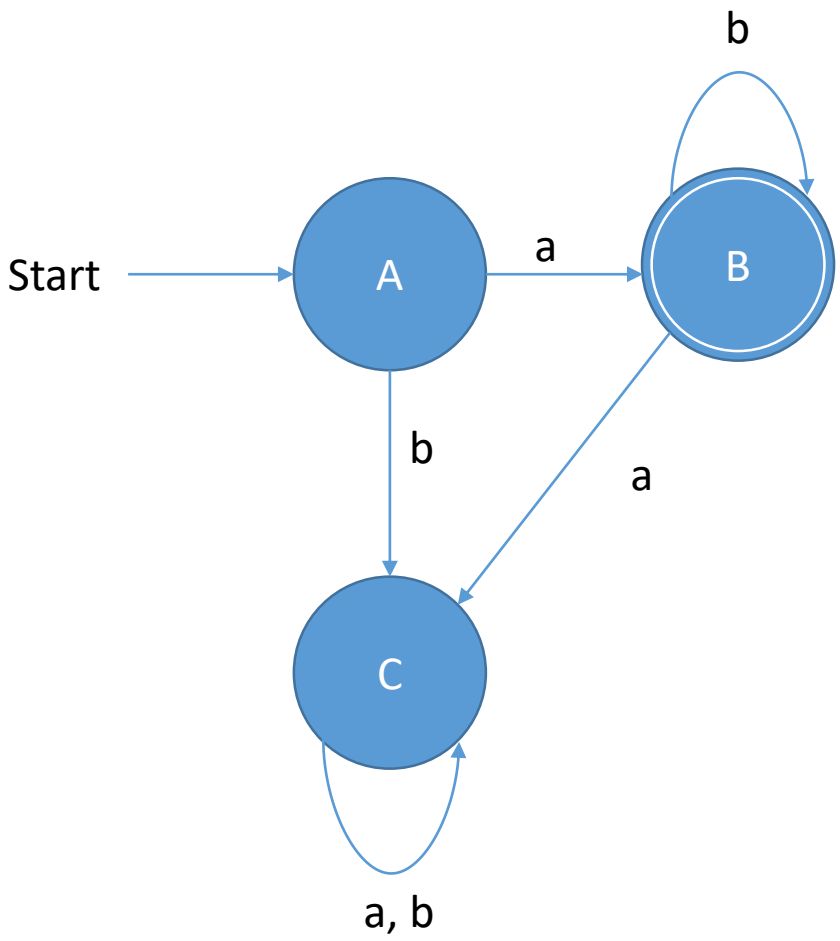
Reverse of DFA/NFA

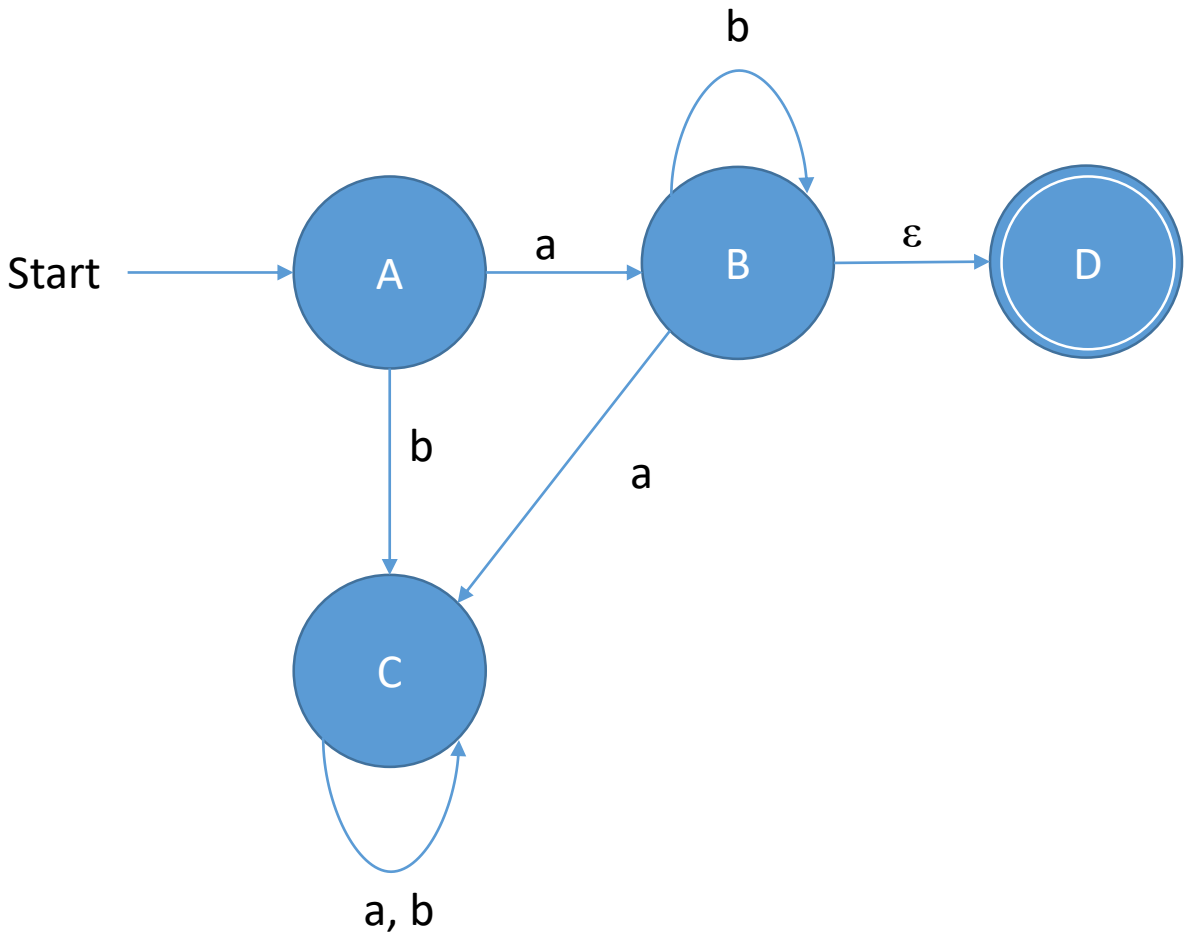
$L_1 = \{w \in \{a,b\}^* \mid w = ab^n \text{ for some } n \geq 0\}$



Reverse of DFA/NFA

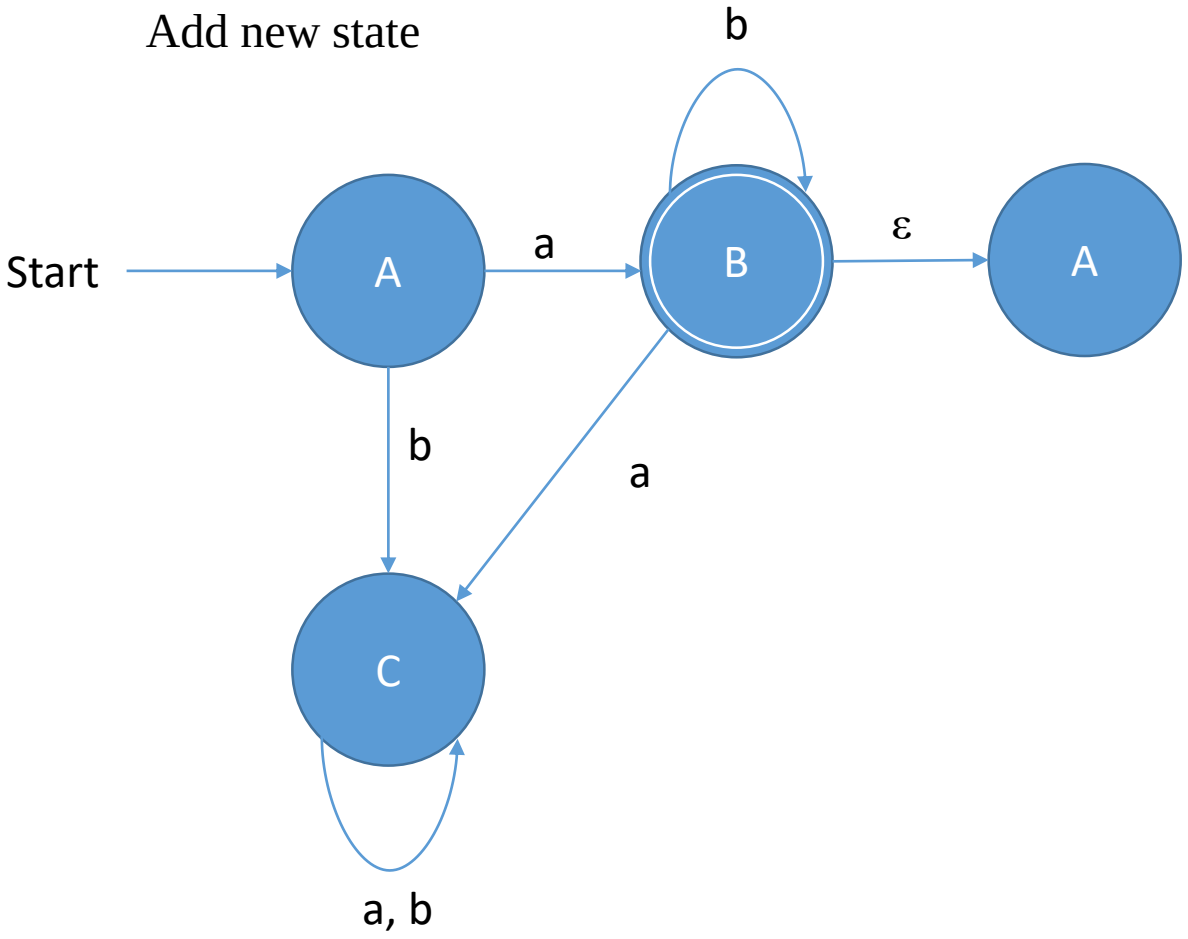
Add new single accepting state



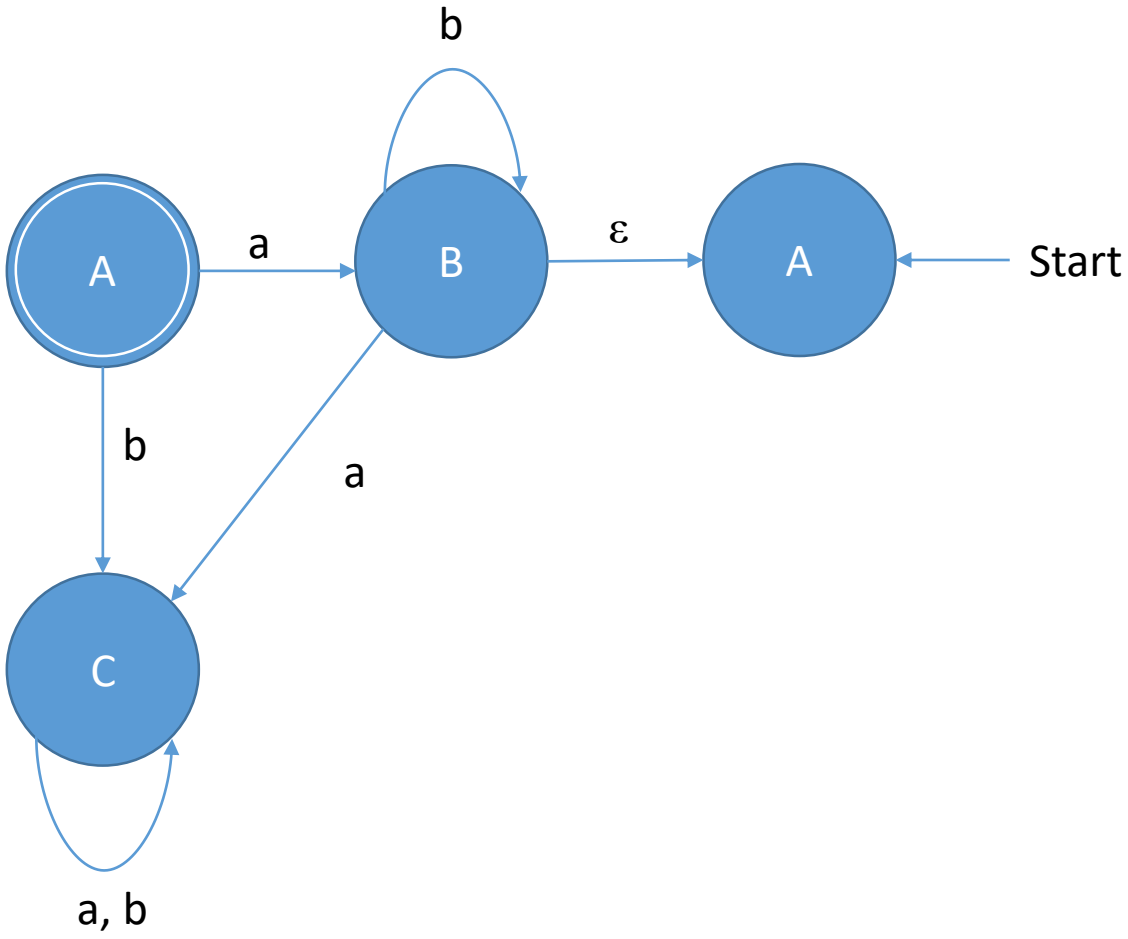


Reverse of DFA/NFA

Add new state

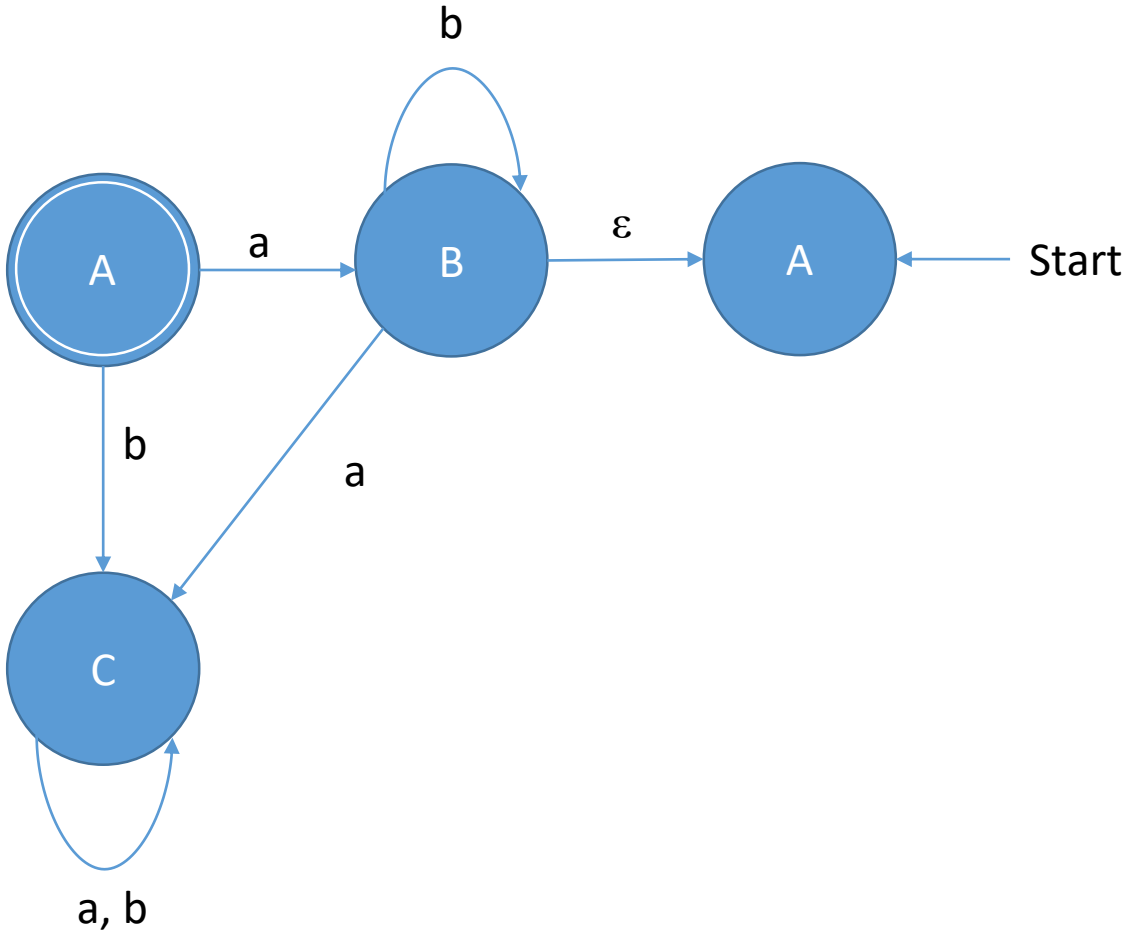


Make start state final state and final state start state

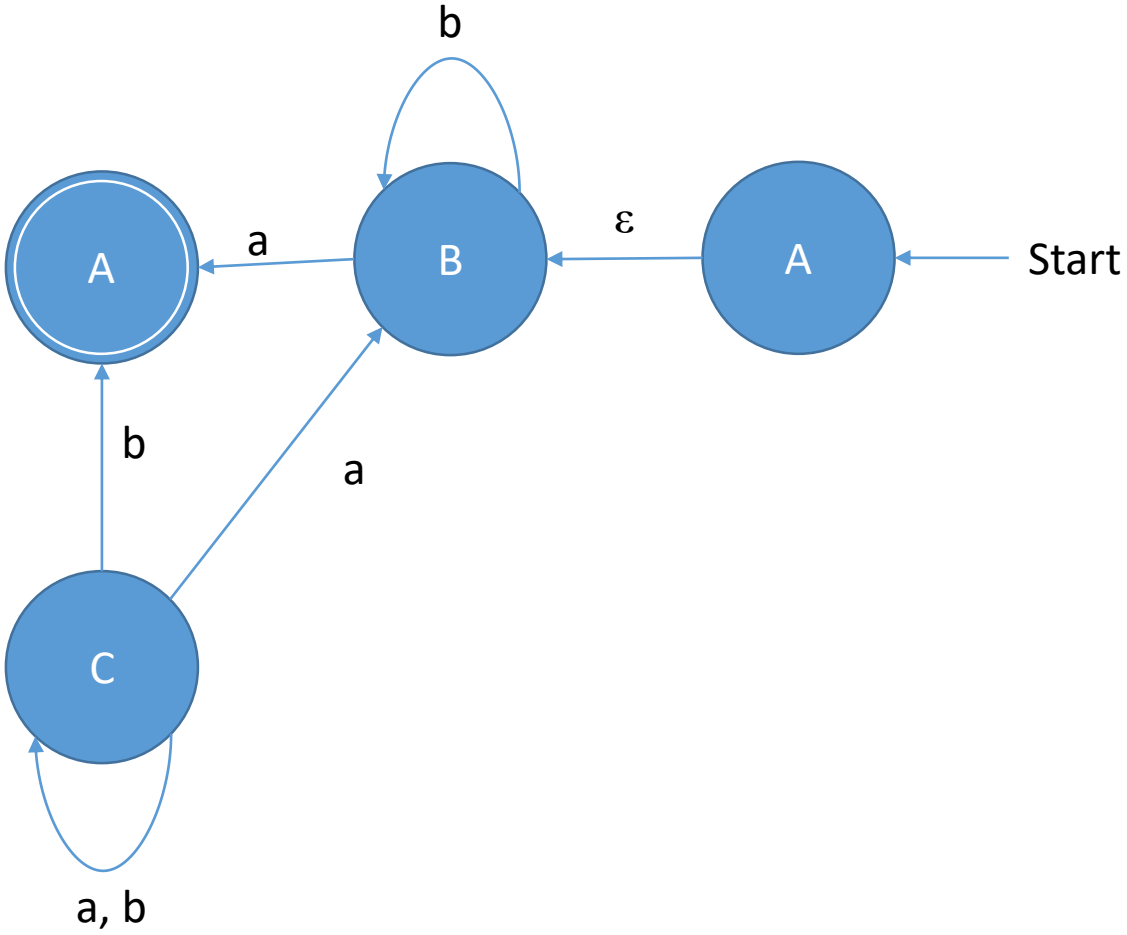


Reverse of DFA/NFA

Make start state final state and final state start state

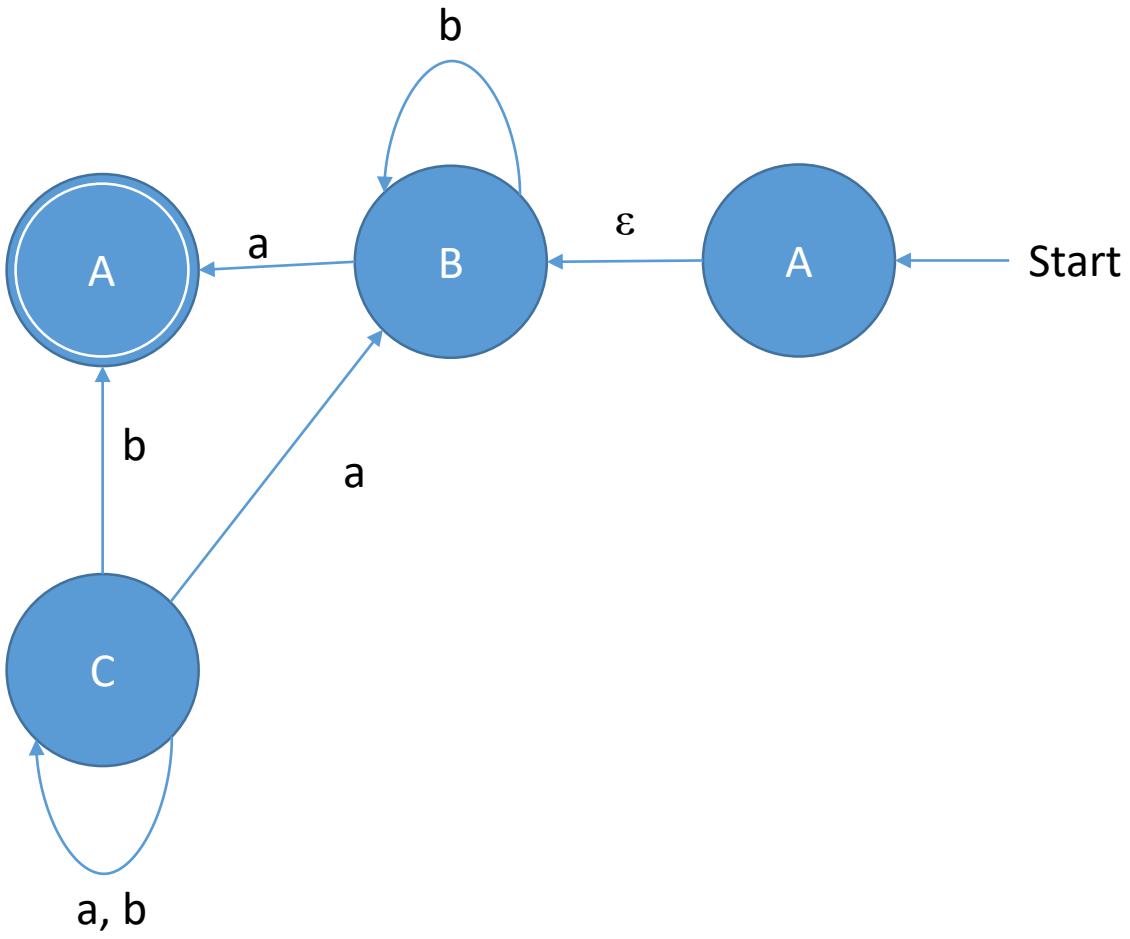


Invert the transition

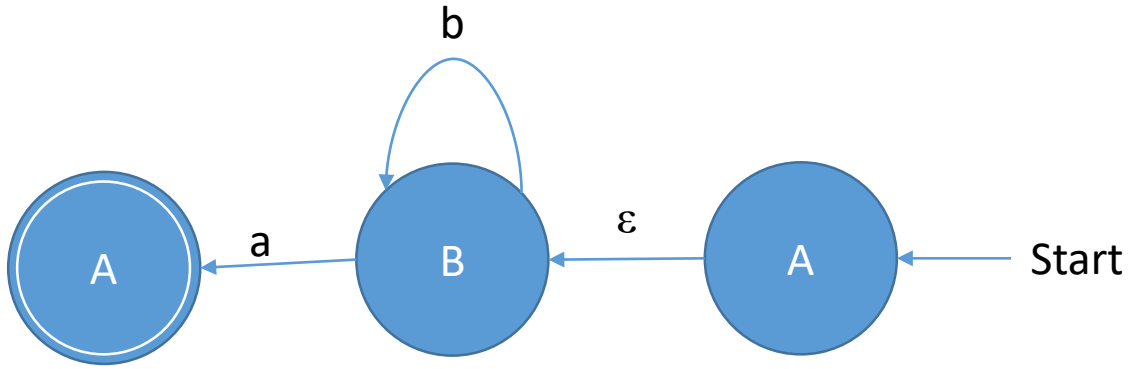


Reverse of DFA/NFA

Invert the transition



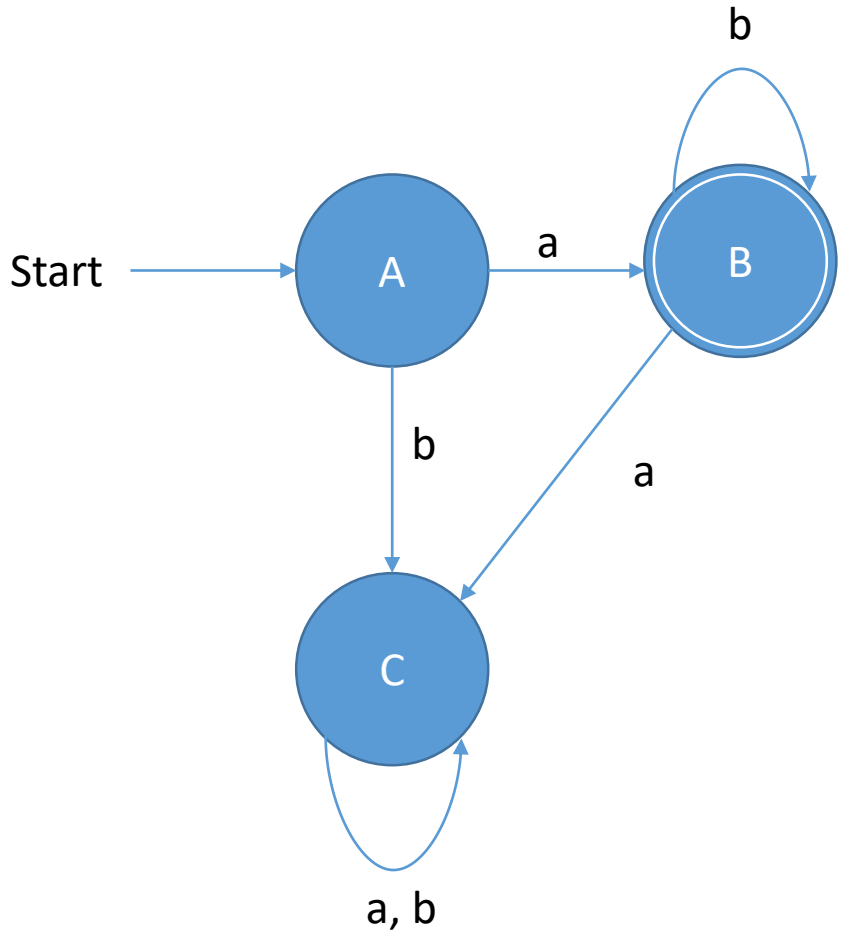
Remove unnecessary

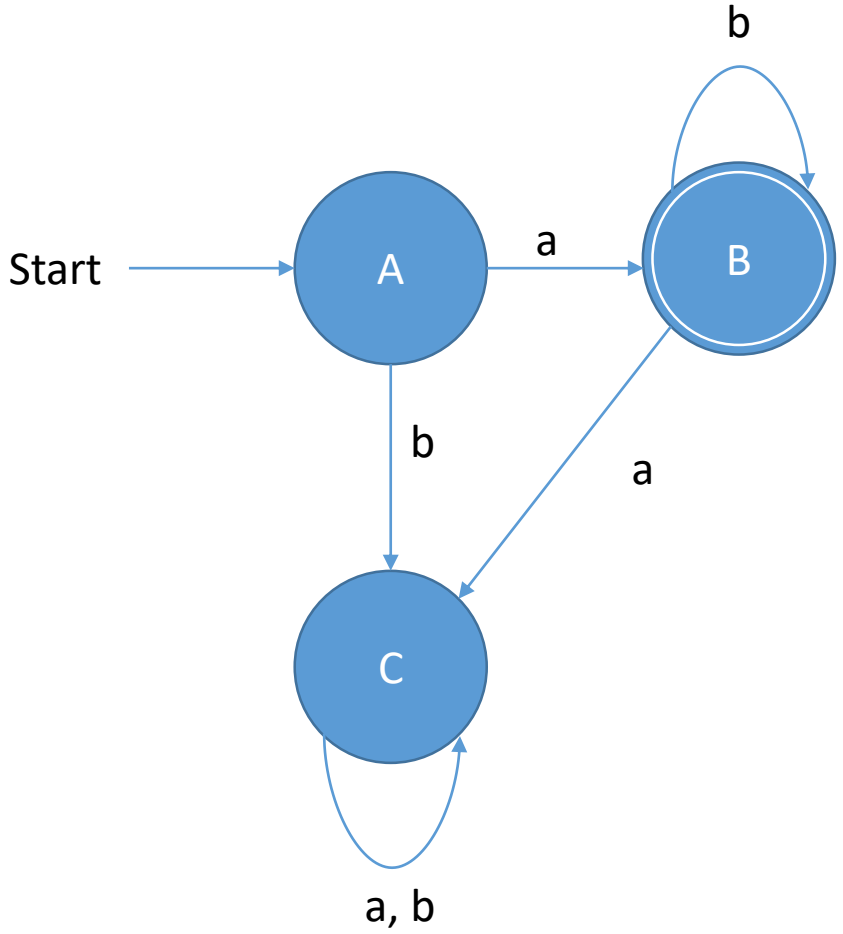


Inversion/Complement of DFA/NFA

$L_1 = \{w \in \{a,b\}^* \mid w = ab^n \text{ for some } n \geq 0\}$

Make all accepting state to non-accepting state also make all non-accepting state to accepting state





Inherent ambiguity

Ambiguity is of 2 types :

a) Removable ambiguity :

E --> E + E | E * E | id

b) Non removable ambiguity (or inherent ambiguity) :

One common example is :

$$L = \{a^m b^m c^n \mid m,n >= 0\} \cup \{a^m b^n c^n \mid m,n >= 0\}$$

S → A | B

A → aCbD

C → aCb|ε

D → cC|c

B → EbFc

E → aE|a

F → bFc| ε

Homomorphism

Homomorphism: A homomorphism on an alphabet is a function that gives a string for each symbol in that alphabet. Example: h(0) = ab; h(1) = ba. It also called substitution function.

L = {w | w ∈ L}

h(L) = {h(w) | w ∈ L}

It's mapping from one alphabet to another.
Mapping $\Sigma_1 = \{0, 1, 2\}$ to $\Sigma_2 = \{a,b)$
The homomorphic function
h(0) = ab and h(1) = a and h(2) = b
012 will be **abab**

A homomorphism of RL is regular.

Inverse Homomorphism

Inverse Homomorphism : Let h be a homomorphism and L a language whose alphabet is the output language of h. h−1 *h*−1 (L) = {w | h(w) is in L}.

L = {w | w ∈ L}

h(L) = {x or h(w) | w ∈ L} = R = {x | x ∈ R}

h^{-1}(R) = {h^{-1}(x) or w | w ∈ L} = L

Mapping Σ_1 = {0, 1} to Σ_2 = {a,b)
h(0) = ab and h(1) = a and h(2) = b

abab will be {00, 012, 120, 1212}

A inverse homomorphism of RL is regular.

Undecidability

There are two types of TMs (based on halting):

Recursive: **TMs that always halt**, no matter accepting or non-accepting ≡ **DECIDABLE** PROBLEMS

Recursively enumerable: **TMs that are guaranteed to halt only on acceptance**. If non-accepting, it may or may not halt (i.e., could loop forever).

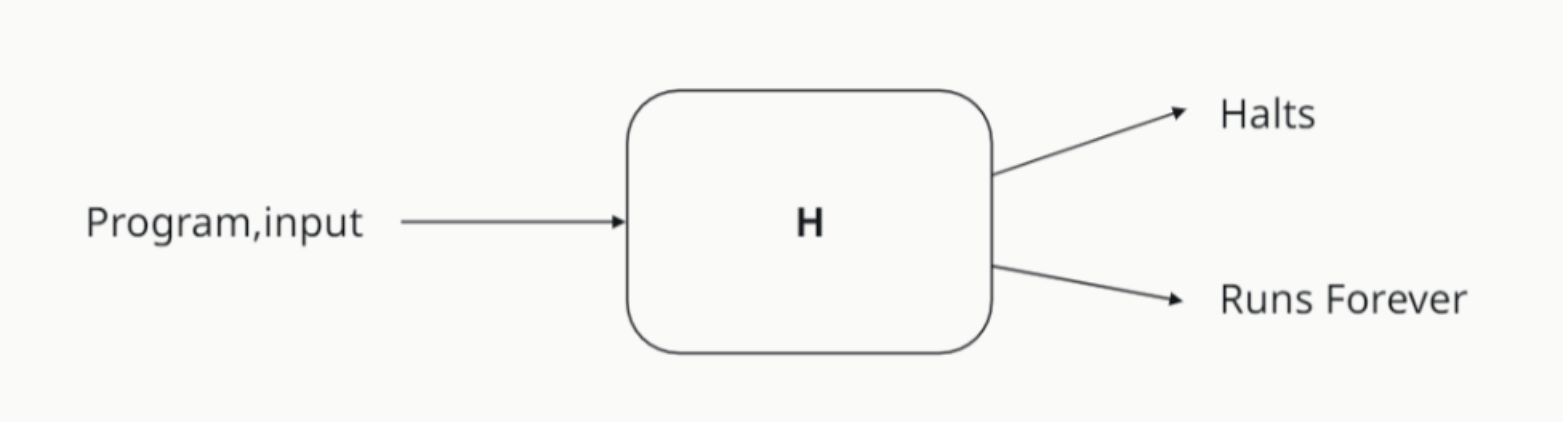
Undecidability:

Undecidable problems are those that are not recursive

Halting Problem

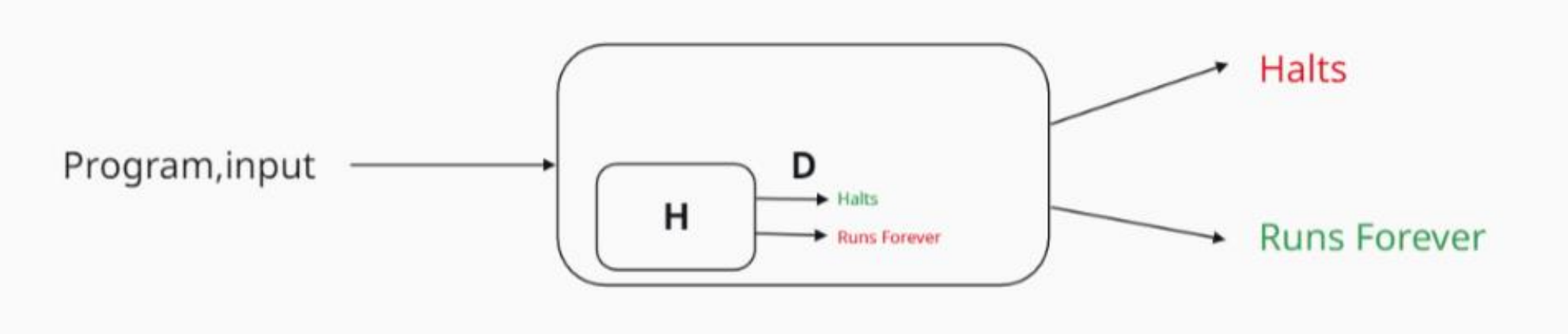
- The Halting Problem asks whether there is a general algorithm that can examine any given computer program and input, and correctly determine whether the program will **eventually stop** (halt) or **loop forever**.

Halting Problem Contd.



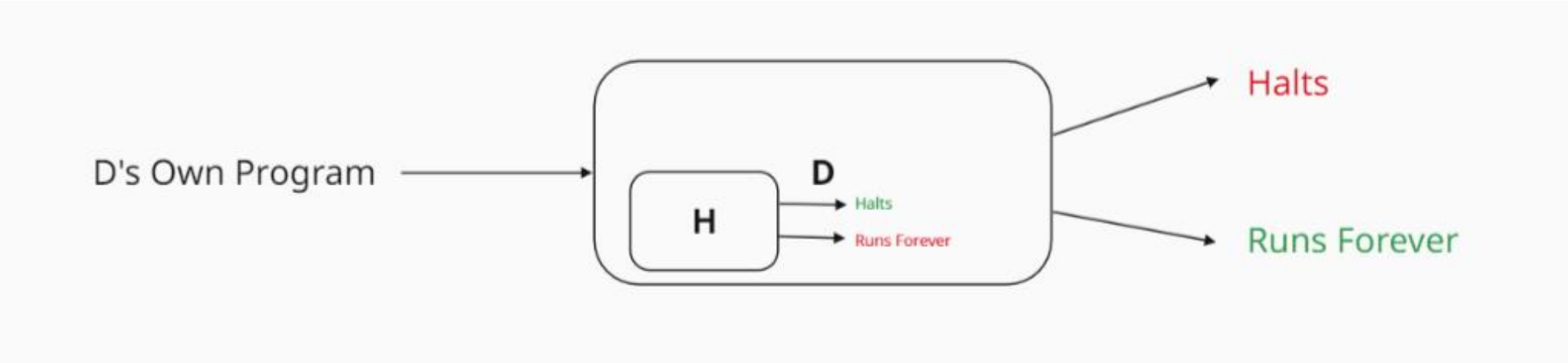
Let's H is always right
Proof by Contradiction!

Halting Problem Contd.



Undecidability

Halting Problem Contd.



H can’t exist!!